PREDICTIVE MAINTENANCE PLATFORM: REAL-TIME MACHINE HEALTH MONITORING, ALERTING, AND MAINTENANCE WORKFLOW FOR MANUFACTURING

A PROJECT REPORT

Submitted by

MOHAN K (310624243099)

in partial fulfillment for the award of the Degree

of

BACHELOR OF ENGINEERING

in

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE



EASWARI ENGINEERING COLLEGE
(AN AUTONOMOUS INSTITUTION)
AFFILIATED TO ANNA UNIVERSITY,
CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled "**PREDICTIVE MAINTENANCE PLATFORM: REAL-TIME MACHINE HEALTH MONITORING, ALERTING, AND MAINTENANCE WORKFLOW FOR MANUFACTURING**" is the Bonafide work of **"MOHAN K (310624243099)"**, who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT

Dr. T. Kalai Selvi

Easwari Engineering College,

Ramapuram, Chennai – 600089.

SUPERVISOR

Ms. D. S. K. Harshitha

Easwari Engineering College,

Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

Certificate Of Viva-Voce Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "Predictive Maintenance Platform: Real-Time Machine Health Monitoring, Alerting, And Maintenance Workflow For Manufacturing" submitted by Mohan K (310624243099) in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering was conducted on 27-10-2026 in the Department of Artificial Intelligence and Data Science, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Date:

ABSTRACT

Manufacturing plants depend on equipment such as lathes, presses, compressors and conveyors. When one of these machines fails without warning, production stops, repair costs rise and delivery dates slip. The problem statement behind this project puts the production lost to unexpected breakdowns at 15 to 40 percent, with repair bills about 30 percent higher than planned maintenance. Plants already collect large volumes of sensor data, but most of them lack the real-time intelligence that turns this data into actions a maintenance team can take.

To address this gap, this project presents a Predictive Maintenance Platform built as an event-driven system on a polyglot stack. Sensors, simulated in this work, publish readings over MQTT. A Spring Boot ingestion service validates every message against a JSON Schema contract and publishes it to Apache Kafka. A second Spring Boot service, the analytics service, stores the readings in a MongoDB time-series collection, evaluates four rules on each sensor (maximum limit, minimum limit, rate of change per minute and sensor offline) and calculates a health score from 0 to 100 for every machine, which maps to the bands healthy, watch, warning and critical. A Node.js and Express 5 service, which forms the MERN part of the stack together with MongoDB and a React 18 front end, consumes the resulting alert and health events, keeps them in MongoDB and pushes them to the browser through Socket.IO.

The platform does not stop at detection. Operators acknowledge or dismiss alerts, create work orders linked to an alert, assign them to technicians, and record the action taken, the parts replaced and the downtime when the job is complete, so every machine builds up a maintenance history. Access is controlled with JSON Web Tokens and four roles (admin, manager, technician and viewer), and the data of each organization is isolated from every other organization. The system is covered by automated tests (95 API tests, 16 ingestion tests and 46 analytics tests), starts locally with a single command, and presents live charts, a health ring and the downtime cost at risk in a responsive and accessible interface.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department **Dr. T. KalaiSelvi**, Department of Artificial Intelligence and Data Science, for the continuous encouragement and support.

We are extremely thankful to our Project Guide **Ms. D. S. K. Harshitha**, Department of Artificial Intelligence and Data Science for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Artificial Intelligence and Data Science for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE No.
	ABSTRACT	i
	ACKNOWLEDGEMENT	ii
	LIST OF TABLES	v
	LIST OF FIGURES	vi
1.	**INTRODUCTION**	1
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	1
	1.3 OBJECTIVE	2
	1.4 SCOPE OF THE PROJECT	3
	1.5 MOTIVATION	4
2.	**LITERATURE SURVEY**	5
	2.1 LITERATURE REVIEW	5
	2.2 EXISTING SYSTEM	6
	2.3 PROPOSED SYSTEM	6
3.	**SYSTEM ANALYSIS**	8
	3.1 SYSTEM REQUIREMENTS	8
	3.1.1 Functional Requirements	8
	3.1.2 Non-Functional Requirements	9
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	10
4.	**METHODOLOGY**	12
	4.1 SYSTEM ARCHITECTURE	12

	4.2 MODULE DESCRIPTION	13
	4.3 CLASS DIAGRAM / DATA MODEL	15
	4.4 SEQUENCE DIAGRAM	16
5.	**RESULTS AND DISCUSSION**	17
	5.1 IMPLEMENTATION OVERVIEW	17
	5.2 KEY FUNCTIONS (CODE EXPLANATION)	18
	5.2.1 Contract-Validated Telemetry Ingestion	18
	5.2.2 Rule Evaluation and Alert Events	19
	5.2.3 Health Score Calculation	20
	5.2.4 Alert Lifecycle and Real-Time Delivery	21
	5.2.5 Authentication and Role-Based Access Control (RBAC)	23
6.	**CONCLUSION & FUTURE WORK**	24
	6.1 CONCLUSION	24
	6.2 FUTURE ENHANCEMENTS	24
	APPENDIX: SOURCE CODE	26
	SCREENSHOTS OF THE PROJECT	40
	REFERENCES	44

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE No.
3.1	Minimum Software Requirements for Development and Production Hosting	10
3.2	Minimum and Recommended Hardware Specifications	11
4.1	Summary of System Modules and Core Responsibilities	14

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE No.
4.1	Layered Event-Driven System Architecture	13
4.2	Data Model of the MongoDB Collections	15
4.3	Reading-to-Work-Order Sequence Diagram	16
1	Fleet Overview Dashboard with Health Summary	40
2	Machine Detail: Health Score Ring and Live Sensor Charts	41
3	Active Alerts Queue with Severity and Status	41
4	Work Order Creation Dialog Linked to an Alert	42
5	Work Order List with Priority, Assignee and Due Date	42
6	Team Management and Role-Based Permissions	43

CHAPTER 1
INTRODUCTION

This chapter deals with the introduction, problem definition, objectives, scope and motivation behind the development of the predictive maintenance platform.

1.1 OVERVIEW

Manufacturing depends on machines that run for long hours under heavy load. Bearings wear, belts slip, lubricants degrade and motors overheat, and each of these faults usually shows up in the sensor data well before the machine stops. Temperature, vibration, current, pressure and speed readings change gradually as a fault develops, so a system that watches them continuously can warn the maintenance team early.

Maintenance in most plants is still either reactive, where a machine is repaired after it fails, or preventive, where parts are replaced on a fixed calendar whether they need it or not. Reactive maintenance causes unplanned downtime and expensive emergency repairs. Calendar-based maintenance wastes parts and labour and still misses faults that appear between two services. Predictive maintenance replaces both with a condition-based approach, in which the actual state of the machine decides when work is done.

This project builds such a platform from end to end. Sensor readings travel through an event-driven pipeline made of MQTT, Apache Kafka and two Spring Boot services. A Node.js and Express API with MongoDB stores assets, alerts and work orders, and a React 18 interface shows the live health of every machine. The platform joins detection and action: an alert raised by the analytics service can be turned into a work order, assigned to a technician and closed with a recorded repair.

1.2 PROBLEM DEFINITION

Manufacturers lose between 15 and 40 percent of production and pay about 30 percent more for repairs when machines break down unexpectedly, yet they lack the real-time intelligence needed to extract actionable insight from the massive volume of sensor data they collect. Three systemic problems stand behind this situation:

1. Data Without Insight: Sensors produce thousands of readings every minute, but the numbers are only stored or shown on a screen. Nobody can watch hundreds of charts all day, so slow degradation goes unnoticed until the machine fails.

2. Late and Unexplained Alarms: Fixed alarm limits in a control room fire only after a value has already crossed a limit and say nothing about the trend. Operators receive alarms without explanation and cannot tell which machine needs attention first or what is wrong with it.

3. Broken Link Between Alert and Action: When an alarm lives in one system and the repair job on paper or in another system, alerts are forgotten, work is duplicated and nobody can show what was done to a machine. Managers cannot judge the cost of downtime because the maintenance history is incomplete.

1.3 OBJECTIVE

The overall goal of this project is to design, develop and test a predictive maintenance platform that turns raw sensor data into timely, explained alerts and tracked maintenance work. The specific objectives are:

• Reliable Telemetry Pipeline: Build an event-driven pipeline from sensor to dashboard (MQTT, Kafka and Spring Boot services) in which every message is validated against a published JSON Schema and invalid messages are kept in a dead-letter topic instead of being lost.

• Rule-Based Detection: Evaluate maximum limit, minimum limit, rate of change per minute and sensor offline rules on each sensor, with a clearing rule that stops alerts from flickering on and off.

• Explained Health Score: Calculate a health score from 0 to 100 for every machine, map it to the bands healthy, watch, warning and critical, and list the reasons in plain language.

• Alert-to-Work-Order Workflow: Support the alert lifecycle (open, acknowledged, in progress, resolved and dismissed) and work orders with assignee, priority, due date, action taken, parts replaced and downtime.

• Live Dashboard: Provide a React 18 interface with live charts, a health ring and a fleet summary of downtime cost at risk, updated over WebSocket without page reloads.

• Secure Multi-Tenant Access: Protect the API with JSON Web Tokens, four roles and strict separation between organizations.

• Repeatable Demonstration: Provide a sensor simulator with injectable faults, automated tests and a one-command start, so that the whole system can be demonstrated on a laptop.

1.4 SCOPE OF THE PROJECT

The scope covers the complete path from a sensor reading to a completed maintenance job. It is divided into the delivered scope and the work that is deliberately left out:

A. In-Scope Engineering Deliverables:

1. Sensor Simulator: A TypeScript tool that publishes realistic readings for every sensor of an organization and can inject faults (overheat, vibration and spike) at a chosen speed.

2. Ingestion Service: A Spring Boot service that subscribes to MQTT, validates messages against the telemetry contract and forwards them to Kafka or to the dead-letter topic.

3. Analytics Service: A Spring Boot service that stores readings in a MongoDB time-series collection, evaluates the four rules and publishes alert and machine health events.

4. REST API and Real-Time Hub: An Express 5 API with Mongoose, a Kafka bridge and a Socket.IO hub.

5. Operations Interface: Management of plants, machines and sensors, live charts, alerts, work orders, maintenance history and team members.

6. Security: JWT access and refresh tokens, bcrypt password hashing and four roles.

7. Quality Assurance: Automated unit, integration and contract tests for the API and both Java services.

B. Out-of-Scope System Boundaries:

1. Learned Failure Prediction: Machine learning models for remaining useful life and learned baselines are left for future work. This version uses explainable rules and a transparent health formula.

2. Physical Equipment Integration and Public Deployment: Connection to real controllers (for example through OPC UA or Modbus), cloud hosting, a custom domain and public legal pages are outside this version, which runs locally with simulated sensors.

1.5 MOTIVATION

The motivation for this project is the gap between the amount of data modern machines produce and the small share of it that helps a maintenance team decide what to do next. A tool that tells a technician which machine needs attention, why, and what has been done to it before, saves production time and repair cost. The project also gives a practical way to apply a full set of current technologies (TypeScript, React, Node.js, MongoDB, Java, Spring Boot, Kafka and MQTT) to one realistic industrial problem, and to show how events, contracts and tests keep such a system understandable as it grows.

CHAPTER 2
LITERATURE SURVEY

This chapter deals with the literature survey of condition-based maintenance and industrial data platforms, the weaknesses of existing maintenance systems, and the proposed platform.

2.1 LITERATURE REVIEW

Maintenance strategies are commonly grouped into corrective, preventive and predictive maintenance (Mobley, 2002). Predictive maintenance relies on measuring the condition of equipment, such as vibration, temperature and electrical current, and acting when the measurements show developing damage. Jardine, Lin and Banjevic (2006) reviewed machinery diagnostics and prognostics for condition-based maintenance and described a maintenance programme as three steps: data acquisition, data processing and maintenance decision-making. The platform in this project follows the same three steps, with sensors and MQTT for acquisition, the analytics service for processing, and alerts with work orders for decision-making.

Lee, Bagheri and Kao (2015) proposed a five-level architecture for cyber-physical systems in Industry 4.0, with the levels connection, conversion, cyber, cognition and configuration. The first levels correspond to collecting data reliably from machines and converting it into information about machine health, which is the part of the architecture that this project implements. The international standard ISO 20816-1 (2016) evaluates machine vibration against defined zones instead of one universal number, which is why every sensor in the platform carries its own configurable limits.

Carvalho et al. (2019) published a systematic review of machine learning methods applied to predictive maintenance. Such methods need a continuous and trustworthy stream of time-stamped sensor data. For this reason the present work first builds the data pipeline, the contracts and an explainable rule layer, which form the base on which learned models can be added later.

On the software side, Kreps, Narkhede and Rao (2011) described Kafka as a distributed, log-based messaging system designed for high-throughput event data, and the MQTT 5.0 specification (OASIS, 2019) defines a lightweight publish and subscribe protocol suited to constrained devices. Newman (2021) explains that event-driven communication keeps services loosely coupled, since a producer does not need to know its consumers. These ideas are the basis of the pipeline described in Chapter 4.

2.2 EXISTING SYSTEM

Maintenance in manufacturing today usually relies on one or more of the following approaches, each with clear weaknesses:

1. Manual Inspection and Calendar-Based Servicing: Technicians walk the plant with checklists, and services follow fixed intervals recorded on paper or in spreadsheets. Faults that develop between two inspections are missed, and parts may be replaced earlier than needed.

2. Control-Room Monitoring and Fixed Alarms: SCADA screens and data historians display live values and raise an alarm when a value crosses a fixed limit. They show what is happening, but the alarm does not say why, does not rank machines by urgency and is not linked to a repair job.

3. Stand-Alone Maintenance Software: Computerized maintenance management systems record work orders and spare parts but have no live connection to the sensors. Someone must notice a problem elsewhere and enter it by hand, which causes delays and duplicate work.

2.3 PROPOSED SYSTEM

To overcome these weaknesses, this project proposes an integrated, event-driven predictive maintenance platform with four main ideas:

1. Contract-First Event Pipeline: Telemetry, alert and machine health messages are described by JSON Schemas. The ingestion service validates every reading, sends valid

readings to the Kafka topic `telemetry.raw` and sends invalid ones, together with the reason, to `telemetry.deadletter`, so no message disappears silently.

2. Explained Health Scoring: The analytics service evaluates four rules on every sensor and turns the result into one health score per machine with up to three plain-language reasons, so operators know which machine to visit first and why.

3. Closed Alert-to-Action Loop: An alert can be acknowledged, dismissed with a reason, or converted into a work order. When the work is complete, the action taken, parts replaced and downtime are recorded, and when the readings recover the alert is resolved by the system itself.

4. Secure Live Multi-Tenant Dashboard: Each organization sees only its own plants. Changes appear on screen as soon as the events arrive through Socket.IO, in an interface that is keyboard accessible, readable on a phone and does not rely on colour alone.

CHAPTER 3
SYSTEM ANALYSIS

This chapter deals with the system analysis of the platform, specifying the functional and non-functional requirements together with the software and hardware environments needed for development and use.

3.1 SYSTEM REQUIREMENTS

3.1.1 Functional Requirements

• FR-1: Authentication and Access Control: The system must authenticate users with JSON Web Tokens signed with HMAC-SHA256 (access token for 15 minutes, refresh token for 7 days in an httpOnly cookie) and enforce role-based access control for four roles: admin, manager, technician and viewer.

• FR-2: Asset Management: The system must let authorized users create, edit and delete plants, machines and sensors, including the sensor type, unit, sampling rate and alert limits (minimum, maximum and rate of change).

• FR-3: Telemetry Ingestion: The system must accept readings over MQTT on the topic `plant/{plantId}/machine/{machineId}/sensor/{sensorId}`, validate them against the telemetry contract and send invalid messages to a dead-letter topic.

• FR-4: Telemetry Storage: The system must store every valid reading in a MongoDB time-series collection that expires data after 30 days.

• FR-5: Rule Evaluation: The system must evaluate maximum limit, minimum limit, rate of change per minute and sensor offline rules, open an alert when a rule is violated and resolve it after three consecutive normal readings.

• FR-6: Health Score: The system must calculate a health score from 0 to 100 for every machine, classify it as healthy (80 or more), watch (50 to 79), warning (20 to 49) or critical (below 20), and list up to three reasons.

• FR-7: Alert Lifecycle: The system must support the alert states OPEN, ACKNOWLEDGED, IN_PROGRESS, RESOLVED and DISMISSED, and require a reason when an alert is dismissed.

• FR-8: Work Orders: The system must let managers create work orders (optionally from an alert), assign them, and let the assignee start and complete them with the action taken, parts replaced and downtime, while keeping a maintenance history per machine.

• FR-9: Live Dashboard: The system must push new readings, machine health changes and alert changes to the browser without a page reload, and show a fleet summary including the downtime cost at risk per hour.

• FR-10: Simulation: The system must include a sensor simulator that publishes plausible readings and can inject faults (overheat, vibration and spike) at an adjustable speed.

3.1.2 Non-Functional Requirements

• NFR-1: Performance and Responsiveness: Live readings must be pushed to the browser over WebSocket as they arrive, list endpoints must be bounded (at most 200 rows) and telemetry must be read from an indexed time-series collection.

• NFR-2: Reliability and Data Integrity: Messages must be validated at the boundary, consumers must retry when a broker is unavailable, and an alert that is already active for a sensor and rule must be updated instead of duplicated.

• NFR-3: Security: Passwords must be hashed with bcrypt, tokens verified on every protected route, input validated with zod, security headers set with helmet, and the records of one organization must never be visible to another organization.

• NFR-4: Usability and Accessibility: The interface must be responsive from phone to desktop, operable with the keyboard, keep a visible focus indicator, avoid conveying status by colour alone, respect the reduced-motion setting and keep text contrast at 4.5:1 or better.

• NFR-5: Maintainability and Testability: Components must communicate through documented contracts, and every service must have automated tests that can run without the other services.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The software and hardware environments needed to develop and run the platform are listed in Tables 3.1 and 3.2:

Table 3.1: Minimum Software Requirements for Development and Production Hosting

Software Layer	**Component / Technology**	**Version**	**Role in Platform**
Operating System	Microsoft Windows 11 / Linux	64-bit	Development and hosting OS
Runtime Environment	Node.js LTS	v22.x	Runs the API, simulator and build tools
Java Runtime	Eclipse Temurin JDK	21	Runs the ingestion and analytics services
Programming Language	TypeScript	6.x (web), 7.x (API)	Type-safe full-stack development
Frontend Framework	React and Vite	React 18.3 / Vite 8	Declarative UI with a fast development server
Backend Web Server	Express.js	v5.x	REST routing, middleware and controllers
Database ODM	Mongoose	v9.x	Schema enforcement and MongoDB queries
Database Engine	MongoDB	v7.0	Documents and time-series telemetry
Streaming Services	Spring Boot with Gradle	4.1 / Gradle 9.7	Ingestion and analytics services
Message Broker	Apache Kafka	3.9 (KRaft mode)	Durable event streaming between services
MQTT Broker	Eclipse Mosquitto	2.0	Receives sensor readings

Real-Time Channel	Socket.IO	v4.8	WebSocket push to the browser
Security and Auth	JSON Web Token and Bcrypt	jsonwebtoken 9.x, bcryptjs 3.x	HMAC-SHA256 tokens and password hashing
Testing	Vitest, Supertest and JUnit	Vitest 5.x, Supertest 7.x	Automated API and service tests
Containers	Docker Compose	Docker Desktop	Runs MongoDB, Kafka and Mosquitto

Table 3.2 : Minimum and Recommended Hardware Specifications

Hardware Component	**Minimum Development Spec**	**Recommended Target**
Host Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 cores, 2.5 GHz)	Intel Core i7 / AMD Ryzen 7 (8 cores, 3.5 GHz)
System Memory (RAM)	8 GB (run without the Java services)	16 GB (full system with Docker)
Persistent Storage	256 GB SSD (20 GB free space)	512 GB NVMe SSD
Network Connectivity	Broadband Internet / LAN (5 Mbps)	Wired LAN (100 Mbps, low latency)
Display Resolution	1366 x 768 WXGA	1920 x 1080 Full HD

CHAPTER 4
METHODOLOGY

This chapter deals with the methodology and design of the platform, illustrating the layered architecture, the module descriptions, the data model of the MongoDB collections, and the sequence of events from a sensor reading to a completed work order.

4.1 SYSTEM ARCHITECTURE

The platform follows a layered, event-driven architecture that keeps concerns separate and lets each service fail, restart and scale on its own. As Figure 4.1 shows, it is divided into five tiers:

1. Device and Messaging Tier: Sensors, simulated by a TypeScript tool in this work, publish JSON readings to an Eclipse Mosquitto MQTT broker on topics of the form `plant/{plantId}/machine/{machineId}/sensor/{sensorId}`.

2. Ingestion Tier: A Spring Boot service subscribes to `plant/#`, rejects payloads that are too large or not valid JSON, validates the rest against `telemetry.schema.json`, checks that the topic ids match the payload ids and publishes the reading to the Kafka topic `telemetry.raw`, keyed by machine. Every rejected message goes to `telemetry.deadletter` together with the reason.

3. Streaming and Analytics Tier: A second Spring Boot service consumes `telemetry.raw` in batches, writes the readings to a MongoDB time-series collection, runs the rule engine and the health calculator, and publishes alert events to `alerts` and machine health events to `machine.health`.

4. Application Tier: An Express 5 service with a Kafka bridge applies the alert and health events to MongoDB through Mongoose, serves the REST API for plants, machines, sensors, alerts, work orders and users, enforces JWT authentication and roles, and pushes changes to browsers through a Socket.IO hub.

5. Presentation Tier: A React 18 single-page application that uses TanStack Query for server data, a Socket.IO client for live updates and hand-written SVG charts.

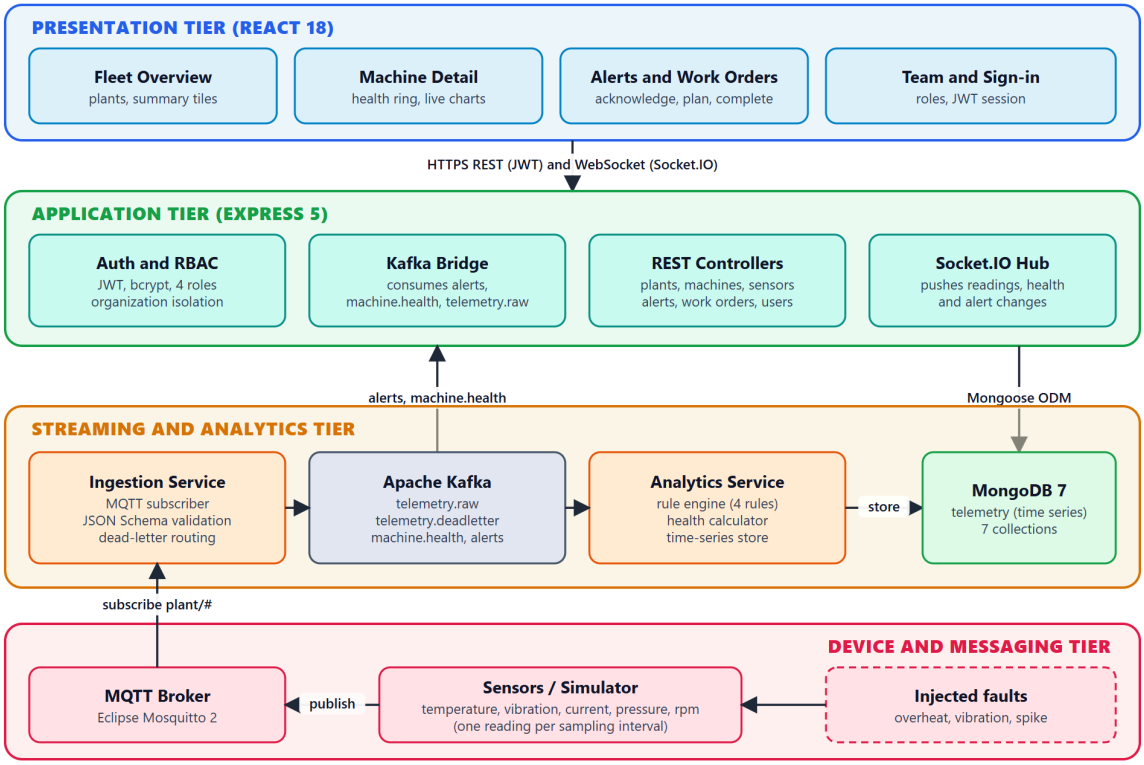


Figure 4.1: Layered Event-Driven System Architecture

4.2 MODULE DESCRIPTION

The platform is made of seven modules that work together to turn a reading into a maintenance action:

4.2.1 Simulator and Device Interface: Logs in to the API, reads the sensors of an organization and publishes a reading for each sensor at its own sampling rate (once per second by default), with optional faults.

4.2.2 Ingestion and Validation Module: Subscribes to MQTT, validates every message against the telemetry contract and routes it to Kafka or to the dead-letter topic.

4.2.3 Analytics Module: Stores readings, evaluates the four rules, calculates the health score of each machine and publishes events.

4.2.4 Alert Management Module: Opens, updates and resolves alerts from analytics events and lets users acknowledge or dismiss them.

4.2.5 Work Order and Maintenance Module: Creates work orders from alerts, assigns them, tracks start and completion and keeps the maintenance history of each machine.

4.2.6 Authentication and Tenant Isolation Module: Registers organizations, signs users in, issues tokens and restricts every query to the organization of the user.

4.2.7 Live Dashboard Module: Shows the fleet summary, machine health, live charts, alerts, work orders and team in the browser.

Table 4.1: Summary of System Modules and Core Responsibilities

Module Code	**Module Name**	**Primary Entities**	**Key Responsibility**
MOD-01	Simulator	Sensor, Reading	Publishes readings and injects faults
MOD-02	Ingestion and Validation	Reading, Dead letter	Schema validation, Kafka routing
MOD-03	Analytics	Telemetry, Machine	Rules, health score, time-series store
MOD-04	Alert Management	Alert	Alert lifecycle, acknowledge, dismiss
MOD-05	Work Orders	WorkOrder	Planning, assignment, completion, history
MOD-06	Auth and Tenancy	Organization, User	Login, JWT, roles, organization isolation
MOD-07	Live Dashboard	Plant, Machine, Sensor	Live charts, health ring, fleet summary

4.3 CLASS DIAGRAM / DATA MODEL

The data is stored in MongoDB and modelled with Mongoose. Figure 4.2 shows the entities, their main attributes and the relationships between them. Every document carries the identifier of its organization, which is the basis of tenant isolation. Telemetry readings are kept in a time-series collection with the identifiers of the plant, machine and sensor as metadata:

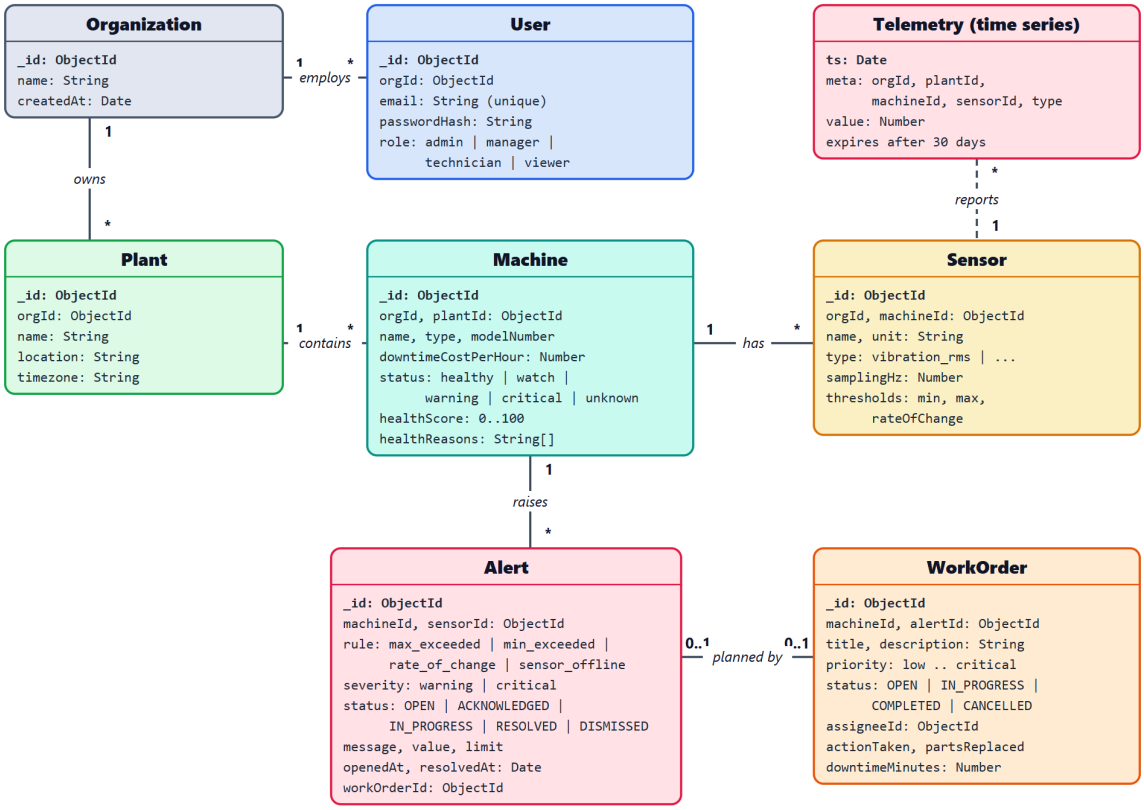


Figure 4.2: Data Model of the MongoDB Collections

4.4 SEQUENCE DIAGRAM

Figure 4.3 shows the runtime sequence of one reading that crosses a limit, from the sensor to a completed work order:

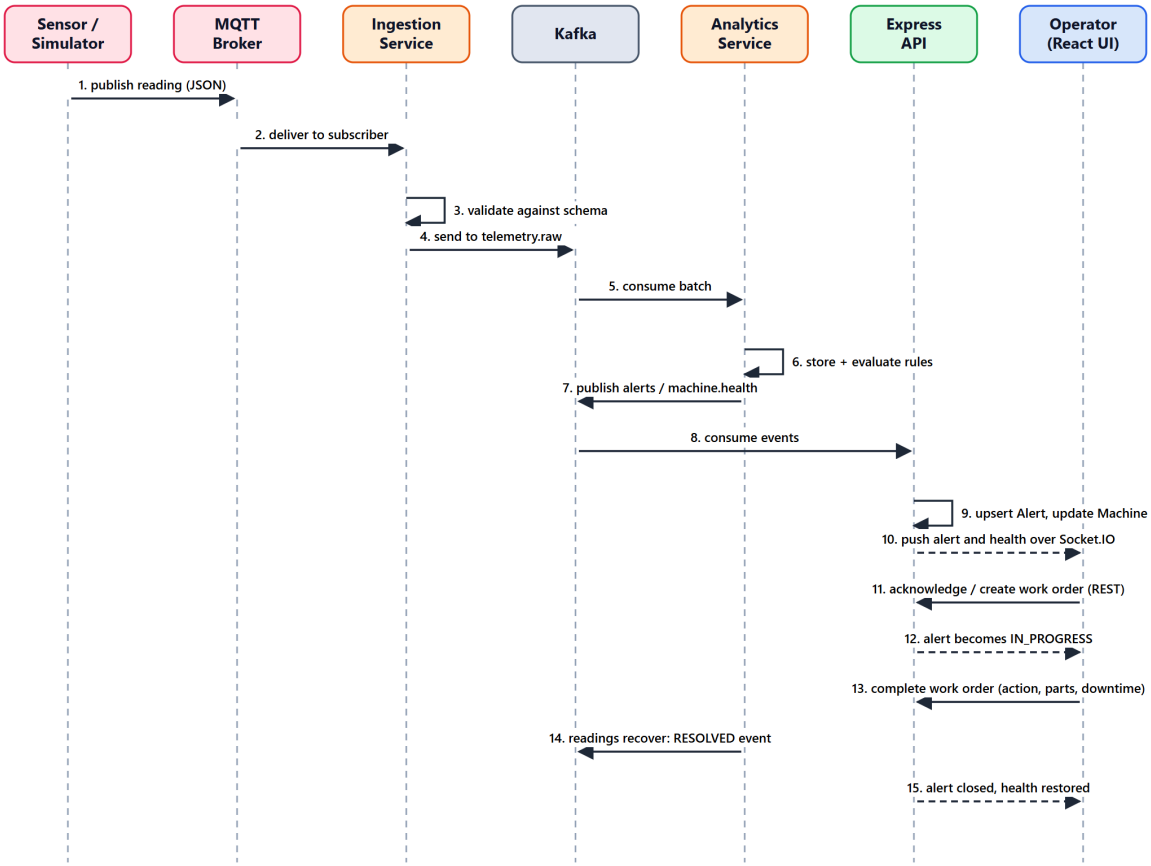


Figure 4.3: Reading-to-Work-Order Sequence Diagram

CHAPTER 5
RESULTS AND DISCUSSION

This chapter deals with the implementation and results of the platform, reviewing the technology configuration and giving algorithmic walkthroughs with code explanations of ingestion, rule evaluation, health scoring, alert delivery and access control.

5.1 IMPLEMENTATION OVERVIEW

The project is a pnpm monorepo with five parts: `apps/api` (Express API), `apps/web` (React front end), `services/ingestion` and `services/analytics` (Spring Boot, built with Gradle), `tools/simulator` (TypeScript) and `contracts` (the JSON Schemas shared by all components). One command, `pnpm start:all`, starts the containers (MongoDB, Kafka and Mosquitto), the API, the web server, both Java services and the simulator.

Frontend Architecture: Built with React 18, TypeScript, React Router and TanStack Query. The access token is kept in memory and a refresh cookie restores the session after a reload. Charts are written as SVG with fixed limit lines, an area fill, a marker on the latest value and diamond markers on readings outside their limits, so a violation is visible without relying on colour. Styling is plain CSS with design tokens and a frosted-glass look, and page changes use short transitions that respect the reduced-motion setting.

Backend Architecture: Built with Express 5 and TypeScript, with Mongoose models for organizations, users, plants, machines, sensors, alerts and work orders. The Kafka bridge uses kafkajs. Input is validated with zod, security headers are set with helmet, and every query is filtered by the organization of the signed-in user.

Testing and Verification: The API has 95 automated tests (Vitest and Supertest against a separate test database), the ingestion service has 16 tests and the analytics service has 46 tests (JUnit), including tests that check the shared example messages against the JSON Schemas. In end-to-end runs with the simulator, a healthy machine shows a score of 100. When the vibration fault is injected, the spindle vibration rises past its limit of 9 mm/s, a critical alert opens, the score falls to 0 and the status becomes critical. Creating a work order

moves the alert to in progress, and once the readings return to normal, three consecutive normal readings resolve the alert.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

5.2.1 Contract-Validated Telemetry Ingestion

The `process` method of the ingestion service is the single entry point for every MQTT message. It rejects oversized payloads, invalid JSON, contract violations and topic mismatches by sending them to the dead-letter topic with a reason, and forwards only valid readings to Kafka:

```
// services/ingestion/src/main/java/com/pm/ingestion/TelemetryProcessor.java
    public void process(String topic, byte[] payload) {
        String text = new String(payload, StandardCharsets.UTF_8);

        if (payload.length > MAX_PAYLOAD_BYTES) {
            deadLetter(topic, text, "payload larger than " + MAX_PAYLOAD_BYTES + "
bytes");
            return;
        }

        JsonNode message;
        try {
            message = mapper.readTree(text);
        } catch (JacksonException e) {
            deadLetter(topic, text, "not valid JSON");
            return;
        }

        List<String> violations = validator.validate(text);
        if (!violations.isEmpty()) {
            deadLetter(topic, text, "contract violation: " + String.join("; ", viol
ations));
            return;
        }

        Matcher matcher = TOPIC.matcher(topic);
        if (!matcher.matches()
                || !matcher.group(1).equals(message.get("plantId").asString())
                || !matcher.group(2).equals(message.get("machineId").asString())
                || !matcher.group(3).equals(message.get("sensorId").asString())) {
            deadLetter(topic, text, "topic does not match payload ids");
            return;
```

```
        }

        sink.send(properties.topics().raw(), message.get("machineId").asString(), t
ext);
        accepted.increment();
    }
```

5.2.2 Rule Evaluation and Alert Events

The rule engine keeps the state of every sensor. A rule opens an alert the first time it is violated and resolves it only after three consecutive readings are back within the limit, which prevents alerts from flickering around a threshold:

```
// services/analytics/src/main/java/com/pm/analytics/RuleEngine.java
    private void update(
            List<AlertEvent> events,
            SensorState state,
            SensorConfig config,
            Reading reading,
            Rule rule,
            boolean violated,
            String severity,
            double value,
            double limit,
            String openMessage,
            String resolvedMessage) {
        SensorState.ActiveCondition active = state.active.get(rule);
        if (violated) {
            if (active == null) {
                state.active.put(rule, new SensorState.ActiveCondition(severity));
                events.add(event(
                        config,
                        reading.plantId(),
                        rule,
                        AlertEvent.OPEN,
                        severity,
                        value,
                        limit,
                        openMessage,
                        reading.ts()));
            } else {
                active.clearStreak = 0;
            }
        } else if (active != null) {
            active.clearStreak++;
            if (active.clearStreak >= CLEAR_AFTER_READINGS) {
```

```
                state.active.remove(rule);
                events.add(event(
                        config,
                        reading.plantId(),
                        rule,
                        AlertEvent.RESOLVED,
                        active.severity,
                        value,
                        limit,
                        resolvedMessage,
                        reading.ts()));
            }
        }
    }
```

5.2.3 Health Score Calculation

The health of a machine is decided by its worst sensor. A sensor outside its limit has full stress, a sensor close to a limit has partial stress, and a rate-of-change or offline condition has medium stress. The score is 100 minus 100 times the worst stress, and the status follows from the score:

```
// services/analytics/src/main/java/com/pm/analytics/HealthCalculator.java
    private static Assessment assess(SensorView view) {
        SensorConfig config = view.config();
        SensorState state = view.state();
        double value = state.lastValue;
        String name = config.name();
        String unit = config.unit();

        double stress = 0;
        String reason = null;

        boolean overMax = config.max() != null
                && (state.active.containsKey(Rule.MAX_EXCEEDED) || value > config.m
ax());
        boolean underMin = config.min() != null
                && (state.active.containsKey(Rule.MIN_EXCEEDED) || value < config.m
in());

        if (overMax) {
            stress = 1;
            reason = name + " is above its maximum of " + Text.number(config.max())
+ " " + unit;
        } else if (underMin) {
            stress = 1;
```

```
            reason = name + " is below its minimum of " + Text.number(config.min())
+ " " + unit;
        } else if (config.min() != null || config.max() != null) {
            double used = utilisation(config, value);
            double proximity = clamp((used - SAFE_FRACTION) / (1 - SAFE_FRACTION))
* AT_LIMIT_STRESS;
            if (proximity > 0) {
                stress = proximity;
                reason = name + " is at " + Math.round(used * 100) + "% of its limi
t";
            }
        }

        if (state.active.containsKey(Rule.RATE_OF_CHANGE) && ATTENTION_STRESS > str
ess) {
            stress = ATTENTION_STRESS;
            reason = name + " is changing faster than its limit";
        }
        if (state.active.containsKey(Rule.SENSOR_OFFLINE) && ATTENTION_STRESS > str
ess) {
            stress = ATTENTION_STRESS;
            reason = name + " has stopped reporting";
        }

        return new Assessment(view, stress, reason);
    }

    static String statusFor(int score) {
        if (score >= 80) return HEALTHY;
        if (score >= 50) return WATCH;
        if (score >= 20) return WARNING;
        return CRITICAL;
    }
```

5.2.4 Alert Lifecycle and Real-Time Delivery

The API consumes the alert events from Kafka. An OPEN event updates the active alert for the same sensor and rule or creates a new one, and a RESOLVED event closes it. Each change is pushed to the signed-in users of the organization through Socket.IO:

```
// apps/api/src/realtime/handlers.ts
export async function applyAlertEvent(event: AlertEvent, hub: Hub) {
  const knownSensor = await Sensor.exists({
    _id: event.sensorId,
    machineId: event.machineId,
    orgId: event.orgId,
  });
```

```
  if (!knownSensor) return null;

  const at = new Date(event.ts);
  const active = {
    orgId: event.orgId,
    sensorId: event.sensorId,
    rule: event.rule,
    status: { $in: ACTIVE_ALERT_STATUSES },
  };

  if (event.state === "OPEN") {
    const existing = await Alert.findOneAndUpdate(
      active,
      {
        severity: event.severity,
        value: event.value,
        limit: event.limit,
        message: event.message,
        lastEventAt: at,
      },
      { returnDocument: "after" },
    );
    const machine = existing ? null : await Machine.findById(event.machineId).selec
t("name");
    const alert =
      existing ??
      (await Alert.create({
        orgId: event.orgId,
        plantId: event.plantId,
        machineId: event.machineId,
        machineName: machine?.name ?? "Unknown machine",
        sensorId: event.sensorId,
        rule: event.rule,
        severity: event.severity,
        value: event.value,
        limit: event.limit,
        message: event.message,
        openedAt: at,
        lastEventAt: at,
      }));
    hub.alertChanged(event.orgId, toAlertDto(alert));
    return alert;
  }

  const resolved = await Alert.findOneAndUpdate(
    active,
    {
```

```
      status: "RESOLVED",
      resolvedAt: at,
      lastEventAt: at,
      resolvedMessage: event.message,
    },
    { returnDocument: "after" },
  );
  if (!resolved) return null;

  hub.alertChanged(event.orgId, toAlertDto(resolved));
  return resolved;
}
```

5.2.5 Authentication and Role-Based Access Control (RBAC)

Protected routes verify the HS256 bearer token and attach the user, organization and role to the request. The role guard then allows only the listed roles:

```
// apps/api/src/auth/middleware.ts
export const requireAuth: RequestHandler = (req, res, next) => {
  const header = req.headers.authorization;
  if (!header?.startsWith("Bearer ")) {
    res.status(401).json({ error: "Missing access token" });
    return;
  }

  try {
    const { sub, orgId, role } = verifyAccessToken(header.slice(7));
    req.auth = { sub, orgId, role };
    next();
  } catch {
    res.status(401).json({ error: "Invalid or expired access token" });
  }
};

export function requireRole(...roles: Role[]): RequestHandler {
  return (req, res, next) => {
    if (!req.auth || !roles.includes(req.auth.role)) {
      res.status(403).json({ error: "Forbidden" });
      return;
    }
    next();
  };
}
```

CHAPTER 6
CONCLUSION & FUTURE WORK

This chapter deals with the evaluation of the platform against its objectives, summarizes the achievements of the project and outlines future enhancements.

6.1 CONCLUSION

The platform shows that an event-driven, contract-first architecture can turn raw sensor readings into explained alerts and tracked maintenance work. By joining detection, health scoring, alert handling and work orders in one system, it removes the gap between noticing a problem and fixing it, and it keeps a history of what was done to each machine.

Key accomplishments of this work include:

• End-to-End Pipeline: Readings flow from sensor to dashboard through MQTT, Kafka, two Spring Boot services, a Node.js API and a React interface, with every message validated against a shared contract.

• Explained Machine Health: A health score from 0 to 100 with plain-language reasons tells operators which machine needs attention first and why.

• Closed Maintenance Loop: Alerts are acknowledged or dismissed with a reason, converted into work orders, completed with the action taken, parts and downtime, and resolved by the system when the readings recover.

• Secure Multi-Tenant Design: JWT authentication, four roles and strict organization isolation protect the data of each customer.

• Tested and Repeatable: 95 API tests and 62 Java tests, a sensor simulator with injectable faults and a one-command start make the system easy to verify and demonstrate.

6.2 FUTURE ENHANCEMENTS

Future versions of the platform can extend it in several directions:

1. Predictive Time-to-Limit Alerts: Estimating from the trend of a sensor when it will reach its limit, so that an alert can be raised before the limit is crossed.

2. Learned Baselines and Machine Learning: Learning the normal behaviour of each machine and applying anomaly detection and remaining-useful-life models on the stored time-series data.

3. Industrial Connectivity: Connecting real equipment through OPC UA or Modbus gateways instead of simulated sensors.

4. Notifications and Mobile Access: Sending alerts by e-mail or messaging apps and offering a mobile application for technicians.

5. Scalable Deployment: Running the services on a container platform with more Kafka partitions, adding browser-level automated tests, and publishing the privacy policy and terms needed for a public launch.

APPENDIX

SOURCE CODE

1. Project Structure

```
predictive-maintenance/
├── apps/
│   ├── api/                    # Express 5 REST API and real-time hub
│   │   ├── src/
│   │   │   ├── auth/           # JWT tokens, requireAuth, requireRole
│   │   │   ├── models/         # Mongoose schemas
│   │   │   ├── realtime/       # Kafka bridge and Socket.IO hub
│   │   │   ├── routes/         # auth, plants, machines, sensors, alerts,
│   │   │   │                   # work orders, users
│   │   │   ├── app.ts          # Express application
│   │   │   └── server.ts       # Server bootstrap
│   │   └── test/               # Vitest and Supertest tests
│   └── web/                    # React 18 single-page application
│       └── src/
│           ├── api/            # Fetch client, hooks, types
│           ├── auth/           # AuthContext
│           ├── components/     # Dialog, HealthCard, LiveChart, ...
│           ├── live/           # Socket.IO context, live readings hook
│           └── pages/          # Plants, Machine, Alerts, Work orders, Team
├── services/
│   ├── ingestion/              # Spring Boot: MQTT to Kafka, validation
│   └── analytics/              # Spring Boot: storage, rules, health score
├── tools/
│   └── simulator/              # TypeScript sensor simulator
├── contracts/                  # JSON Schemas and example messages
├── infra/mosquitto/            # MQTT broker configuration
├── scripts/dev.mjs             # One-command start for the whole system
├── compose.yaml                # MongoDB, Kafka, Mosquitto containers
└── package.json                # pnpm workspace
```

2. Installation and Run Commands

```
# Install all JavaScript dependencies
pnpm install

# Start MongoDB, Kafka and the MQTT broker
docker compose up -d

# Start everything (API, web, ingestion, analytics and simulator)
```

```
pnpm start:all

# Start without the Java services
pnpm start:core

# Run a simulator fault (vibration) at 10 times speed
pnpm --filter @pm/simulator start --fault "CNC Lathe 1=vibration" --speed 10

# Run the automated tests
pnpm --filter @pm/api test
cd services/ingestion && gradlew test
cd services/analytics && gradlew test
```

3. Environment Variables

```
# .env (local development only; replace every placeholder)
MONGO_ROOT_USER=<mongo user>
MONGO_ROOT_PASSWORD=<mongo password>
MONGO_URI=mongodb://<user>:<password>@localhost:27017/predictive_maintenance?authSo
urce=admin
KAFKA_BROKERS=localhost:9092
MQTT_URL=mqtt://localhost:1883
JWT_ACCESS_SECRET=<random string of at least 32 characters>
JWT_REFRESH_SECRET=<a different random string of at least 32 characters>

# Simulator: an existing user of the organization to simulate
API_URL=http://localhost:4000
SIM_EMAIL=<user e-mail>
SIM_PASSWORD=<password of that user>
```

4. Infrastructure: Docker Compose

```
// compose.yaml
  mongo:
    image: mongo:7.0
    ports:
      - "27017:27017"
    environment:
      MONGO_INITDB_ROOT_USERNAME: ${MONGO_ROOT_USER:?Set MONGO_ROOT_USER in .env}
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_ROOT_PASSWORD:?Set MONGO_ROOT_PASSWORD in
.env}
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]
```

```
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 20s
    restart: unless-stopped

  ...

  mosquitto:
    image: eclipse-mosquitto:2.0.20
    ports:
      - "1883:1883"
    volumes:
      - ./infra/mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - mosquitto-data:/mosquitto/data
    restart: unless-stopped
```

5. Telemetry Contract (JSON Schema)

```
// contracts/telemetry.schema.json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://predictive-maintenance.local/contracts/telemetry.schema.json",
  "title": "TelemetryReading",
  "description": "One reading from one sensor. Published to MQTT and Kafka topic te
lemetry.raw.",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "schemaVersion",
    "orgId",
    "plantId",
    "machineId",
    "sensorId",
    "type",
    "unit",
    "value",
    "ts"
  ],
  "properties": {
    "schemaVersion": { "const": 1 },
    "orgId": { "type": "string", "pattern": "^[a-f0-9]{24}$" },
    "plantId": { "type": "string", "pattern": "^[a-f0-9]{24}$" },
    "machineId": { "type": "string", "pattern": "^[a-f0-9]{24}$" },
    "sensorId": { "type": "string", "pattern": "^[a-f0-9]{24}$" },
    "type": {
```

```
      "enum": ["vibration_rms", "temperature", "current", "pressure", "rpm"]
    },
    "unit": { "type": "string", "minLength": 1, "maxLength": 20 },
    "value": { "type": "number" },
    "ts": { "type": "string", "format": "date-time" }
  }
}
```

6. Telemetry Validator

```
// services/ingestion/src/main/java/com/pm/ingestion/TelemetryValidator.java
package com.pm.ingestion;

import com.networknt.schema.Error;
import com.networknt.schema.InputFormat;
import com.networknt.schema.Schema;
import com.networknt.schema.SchemaRegistry;
import com.networknt.schema.SchemaRegistryConfig;
import com.networknt.schema.SpecificationVersion;
import java.io.IOException;
import java.io.InputStream;
import java.util.List;
import org.springframework.core.io.ClassPathResource;
import org.springframework.stereotype.Component;

@Component
public class TelemetryValidator {

    private static final String SCHEMA_RESOURCE = "telemetry.schema.json";

    private final Schema schema;

    public TelemetryValidator() throws IOException {
        SchemaRegistry registry = SchemaRegistry.withDefaultDialect(
                SpecificationVersion.DRAFT_2020_12,
                builder -> builder.schemaRegistryConfig(
                        SchemaRegistryConfig.builder().formatAssertionsEnabled(tru
e).build()));
        try (InputStream in = new ClassPathResource(SCHEMA_RESOURCE).getInputStream
()) {
            this.schema = registry.getSchema(in, InputFormat.JSON);
        }
    }

    /** Returns the violations of the telemetry contract; empty when the message is
valid. */
```

```
    public List<String> validate(String json) {
        return schema.validate(json, InputFormat.JSON).stream().map(Error::getMessa
ge).toList();
    }
}
```

7. Time-Series Telemetry Storage

```
// services/analytics/src/main/java/com/pm/analytics/TelemetryStore.java
    private void ensureCollection() {
        if (collectionReady) {
            return;
        }
        if (!mongo.collectionExists(COLLECTION)) {
            mongo.createCollection(
                    COLLECTION,
                    CollectionOptions.timeSeries(
                            "ts",
                            options -> options
                                    .metaField("meta")
                                    .granularity(Granularity.SECONDS)
                                    .expireAfter(RETENTION)));
        }
        collectionReady = true;
    }

    static Document toDocument(Reading reading) {
        return new Document("ts", Date.from(reading.ts()))
                .append(
                        "meta",
                        new Document("orgId", reading.orgId())
                                .append("plantId", reading.plantId())
                                .append("machineId", reading.machineId())
                                .append("sensorId", reading.sensorId())
                                .append("type", reading.type()))
                .append("value", reading.value());
    }
```

8. Kafka Reading Consumer

```
// services/analytics/src/main/java/com/pm/analytics/ReadingConsumer.java
    void onMessages(List<String> payloads) {
        List<Reading> readings = new ArrayList<>(payloads.size());
        for (String payload : payloads) {
            parser.parse(payload).ifPresentOrElse(readings::add, skipped::incremen
```

```
t);
        }

        readings.forEach(monitor::onReading);
        store.saveAll(readings);
        processed.increment(readings.size());
    }
```

9. Mongoose Model: Machine

```
// apps/api/src/models/machine.ts
const machineSchema = new Schema(
  {
    orgId: {
      type: Schema.Types.ObjectId,
      ref: "Organization",
      required: true,
      index: true,
    },
    plantId: {
      type: Schema.Types.ObjectId,
      ref: "Plant",
      required: true,
      index: true,
    },
    name: { type: String, required: true, trim: true },
    type: { type: String, required: true, trim: true },
    modelNumber: { type: String, trim: true, default: "" },
    installedAt: { type: Date },
    downtimeCostPerHour: { type: Number, min: 0, default: 0 },
    status: { type: String, enum: MACHINE_STATUSES, default: "unknown" },
    healthScore: { type: Number, min: 0, max: 100, default: null },
    healthReasons: { type: [String], default: [] },
    healthUpdatedAt: { type: Date },
  },
  { timestamps: true },
);
```

10. Login and Token Generation

```
// apps/api/src/routes/auth.ts
authRouter.post("/login", async (req, res) => {
  const parsed = loginSchema.safeParse(req.body);
  if (!parsed.success) {
    res
```

```
      .status(400)
      .json({ error: "Invalid input", issues: parsed.error.issues });
    return;
  }
  const { email, password } = parsed.data;

  const user = await User.findOne({ email });
  const passwordOk = await bcrypt.compare(
    password,
    user ? user.passwordHash : DUMMY_HASH,
  );
  if (!user || !passwordOk) {
    res.status(401).json({ error: "Invalid email or password" });
    return;
  }

  const payload = payloadFor(user);
  setRefreshCookie(res, payload);
  res.json({
    accessToken: signAccessToken(payload),
    user: {
      id: user.id,
      email: user.email,
      role: user.role,
      orgId: payload.orgId,
    },
  });
});

export function signAccessToken(payload: TokenPayload) {
  return jwt.sign(payload, config.JWT_ACCESS_SECRET, {
    algorithm: "HS256",
    expiresIn: "15m",
  });
}
```

11. Alert Acknowledge API

```
// apps/api/src/routes/alerts.ts
alertsRouter.post(
  "/:id/acknowledge",
  requireRole("admin", "manager", "technician"),
  async (req, res) => {
    const { orgId, sub } = req.auth!;
    const { id } = req.params;
    if (!mongoose.isValidObjectId(id)) {
```

```
      res.status(404).json({ error: "Alert not found" });
      return;
    }

    const alert = await Alert.findOneAndUpdate(
      { _id: id, orgId, status: "OPEN" },
      {
        status: "ACKNOWLEDGED",
        acknowledgedAt: new Date(),
        acknowledgedByEmail: await emailOf(sub),
      },
      { returnDocument: "after" },
    );
    if (!alert) {
      const exists = await Alert.exists({ _id: id, orgId });
      res
        .status(exists ? 409 : 404)
        .json({ error: exists ? "Only open alerts can be acknowledged" : "Alert not
found" });
      return;
    }

    getHub().alertChanged(orgId, toAlertDto(alert));
    res.json(toAlertDto(alert));
  },
);
```

12. Create Work Order API

```
// apps/api/src/routes/workOrders.ts
workOrdersRouter.post("/", requireRole("admin", "manager"), async (req, res) => {
  const { orgId, sub } = req.auth!;
  const parsed = createSchema.safeParse(req.body);
  if (!parsed.success) {
    res.status(400).json({ error: "Invalid input", issues: parsed.error.issues });
    return;
  }
  const input = parsed.data;

  const machine = await Machine.findOne({ _id: input.machineId, orgId });
  if (!machine) {
    res.status(404).json({ error: "Machine not found" });
    return;
  }

  let assignee = null;
```

```
  if (input.assigneeId) {
    assignee = await findAssignee(orgId, input.assigneeId);
    if (!assignee) {
      res.status(400).json({ error: "The assignee must be a technician, manager or
admin of your organization" });
      return;
    }
  }

  if (input.alertId) {
    const linkable = await Alert.exists({
      _id: input.alertId,
      orgId,
      machineId: machine._id,
      status: { $in: ["OPEN", "ACKNOWLEDGED"] },
      workOrderId: { $exists: false },
    });
    if (!linkable) {
      res.status(409).json({ error: "That alert is not open or already has a work o
rder" });
      return;
    }
  }

  const creator = await User.findById(sub).select("email");
  const order = await WorkOrder.create({
    orgId,
    plantId: machine.plantId,
    machineId: machine._id,
    machineName: machine.name,
    alertId: input.alertId,
    title: input.title,
    description: input.description,
    priority: input.priority,
    assigneeId: assignee?._id,
    assigneeEmail: assignee?.email,
    createdByEmail: creator?.email ?? "unknown",
    dueAt: input.dueAt,
  });

  if (input.alertId) {
    const alert = await Alert.findOneAndUpdate(
      { _id: input.alertId, orgId },
      { status: "IN_PROGRESS", workOrderId: order._id },
      { returnDocument: "after" },
    );
    if (alert) getHub().alertChanged(orgId, toAlertDto(alert));
```

```
  }

  res.status(201).json(toWorkOrderDto(order));
});
```

13. Kafka Bridge Message Handler

```
// apps/api/src/realtime/consumer.ts
export async function handleMessage(topic: string, value: string, hub: Hub) {
  let json: unknown;
  try {
    json = JSON.parse(value);
  } catch {
    console.warn(`Skipping a message on ${topic}: not valid JSON`);
    return;
  }

  if (topic === TOPICS.health) {
    const parsed = healthEventSchema.safeParse(json);
    if (!parsed.success) {
      console.warn(`Skipping an invalid ${topic} message: ${parsed.error.message}
`);
      return;
    }
    await applyHealthEvent(parsed.data, hub);
  } else if (topic === TOPICS.alerts) {
    const parsed = alertEventSchema.safeParse(json);
    if (!parsed.success) {
      console.warn(`Skipping an invalid ${topic} message: ${parsed.error.message}
`);
      return;
    }
    await applyAlertEvent(parsed.data, hub);
  } else if (topic === TOPICS.telemetry) {
    const parsed = readingSchema.safeParse(json);
    if (parsed.success) hub.reading(parsed.data);
  }
}
```

14. Socket.IO Authentication and Hub

```
// apps/api/src/realtime/socket.ts
  io.use((socket, next) => {
    const token: unknown = socket.handshake.auth?.token;
    if (typeof token !== "string") {
```

```
      next(new Error("unauthorized"));
      return;
    }
    try {
      const { orgId } = verifyAccessToken(token);
      socket.data.orgId = orgId;
      next();
    } catch {
      next(new Error("unauthorized"));
    }
  });

  ...

  const hub: Hub = {
    healthChanged(orgId, payload) {
      io.to(`org:${orgId}`).emit("machine:health", payload);
    },
    alertChanged(orgId, payload) {
      io.to(`org:${orgId}`).emit("alert", payload);
    },
    reading(reading) {
      const room = `machine:${reading.machineId}`;
      if ((io.sockets.adapter.rooms.get(room)?.size ?? 0) === 0) return;
      io.to(room).emit("reading", {
        sensorId: reading.sensorId,
        value: reading.value,
        ts: reading.ts,
      });
    },
  };
```

15. React Live Readings Hook

```
// apps/web/src/live/useLiveReadings.ts
export function useLiveReadings(machineId: string) {
  const { socket } = useLive();
  const history = useTelemetry(machineId, WINDOW_MINUTES);
  const [live, setLive] = useState<Record<string, Point[]>>({});

  useEffect(() => {
    setLive({});
  }, [machineId]);

  useEffect(() => {
    if (!socket || machineId === "") return;
```

```
    const subscribe = () => socket.emit("machine:subscribe", machineId);
    const onReading = (reading: LiveReading) => {
      const t = Date.parse(reading.ts);
      setLive((previous) => {
        const kept = (previous[reading.sensorId] ?? []).filter((p) => p.t >= t - WI
NDOW_MS);
        return { ...previous, [reading.sensorId]: [...kept, { t, v: reading.value
}] };
      });
    };

    subscribe();
    socket.on("connect", subscribe);
    socket.on("reading", onReading);
    return () => {
      socket.off("connect", subscribe);
      socket.off("reading", onReading);
      socket.emit("machine:unsubscribe", machineId);
    };
  }, [socket, machineId]);
```

16. Simulator Fault Model

```
// tools/simulator/src/signals.ts
export function nextValue(
  sensor: Sensor,
  machineName: string,
  elapsedMinutes: number,
  faults: Fault[],
): number {
  const base = baselineOf(sensor);
  let value =
    base +
    base * 0.02 * Math.sin(elapsedMinutes / 8) +
    gaussian() * Math.abs(base) * 0.01;
  const reference = sensor.thresholds.max ?? Math.abs(base) * 1.8;

  for (const fault of faults) {
    if (fault.machineName !== machineName) continue;
    if (fault.kind === "overheat" && sensor.type === "temperature") {
      value += reference * 0.04 * elapsedMinutes;
    }
    if (fault.kind === "vibration" && sensor.type === "vibration_rms") {
      value *= 1 + 0.15 * elapsedMinutes;
    }
```

```
    if (fault.kind === "spike" && Math.random() < 0.02) {
      value += reference * 0.6;
    }
  }
  return Math.round(value * 1000) / 1000;
}
```

17. REST API Request and Response Examples

```
// Request: POST /api/auth/login
{
  "email": "admin@example.com",
  "password": "<password>"
}

// Response: 200 OK (the refresh token is set as an httpOnly cookie)
{
  "accessToken": "<JWT, valid for 15 minutes>",
  "user": { "id": "...", "email": "admin@example.com", "role": "admin", "orgId":
"..." }
}

// Request: GET /api/alerts?status=active   (Authorization: Bearer <token>)
// Response: 200 OK
[
  {
    "id": "...",
    "machineName": "CNC Lathe 1",
    "rule": "max_exceeded",
    "severity": "critical",
    "status": "OPEN",
    "value": 9.4,
    "limit": 9,
    "message": "Spindle vibration is 9.4 mm/s, above its maximum of 9 mm/s",
    "openedAt": "2026-10-26T04:07:38.000Z",
    "workOrderId": null
  }
]

// Request: POST /api/work-orders   (roles: admin, manager)
{
  "machineId": "...",
  "alertId": "...",
  "title": "Investigate spindle vibration on CNC Lathe 1",
  "priority": "critical",
  "assigneeId": "...",
```

```
  "dueAt": "2026-10-28"
}

// Response: 201 Created   (the alert changes to IN_PROGRESS)
{
  "id": "...",
  "machineName": "CNC Lathe 1",
  "status": "OPEN",
  "priority": "critical",
  "assigneeEmail": "technician@example.com",
  "createdByEmail": "manager@example.com"
}
```

SCREENSHOTS OF THE PROJECT

Figure 1: Fleet Overview Dashboard with Health Summary

Displays the fleet summary tiles (machines monitored, machines that need attention, active alerts and downtime cost at risk per hour), the count of machines in each health band and the list of plants:

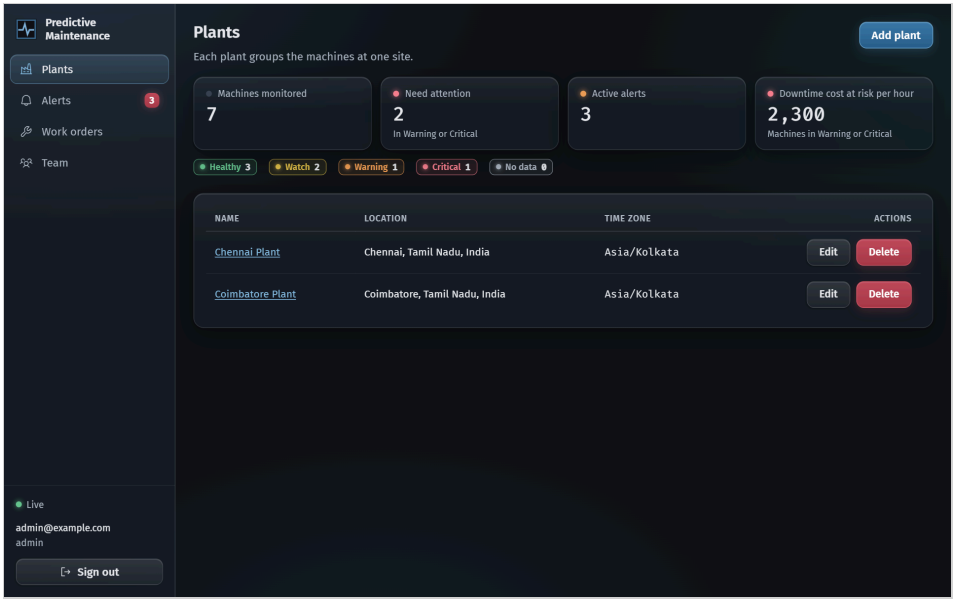


Figure 2: Machine Detail: Health Score Ring and Live Sensor Charts

Shows the health score ring with its status and the reasons behind it, and live charts per sensor with limit lines, minimum, average and maximum values, a marker on the latest reading and markers on readings that cross a limit:

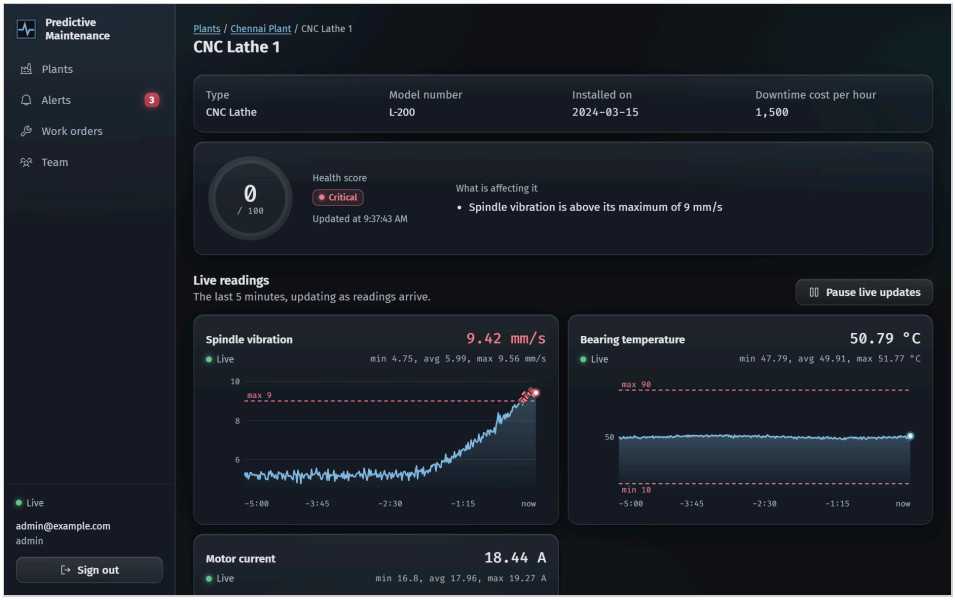


Figure 3: Active Alerts Queue with Severity and Status

Lists the active alerts with severity, message, machine, status and time opened, with actions to acknowledge, create a work order or dismiss according to the role of the user:

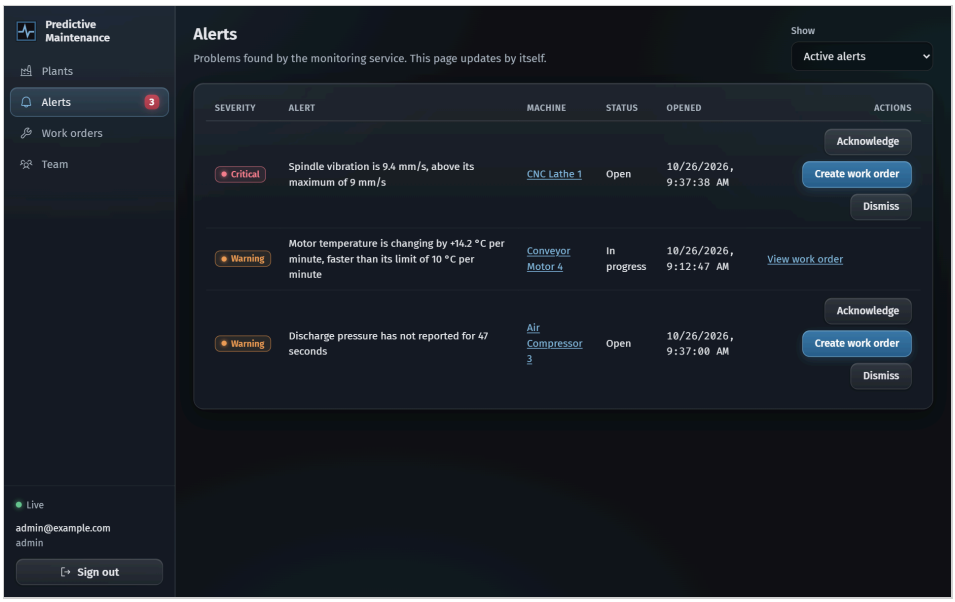


Figure 4: Work Order Creation Dialog Linked to an Alert

The dialog for planning maintenance from an alert, with title, details, priority, due date and assignee:

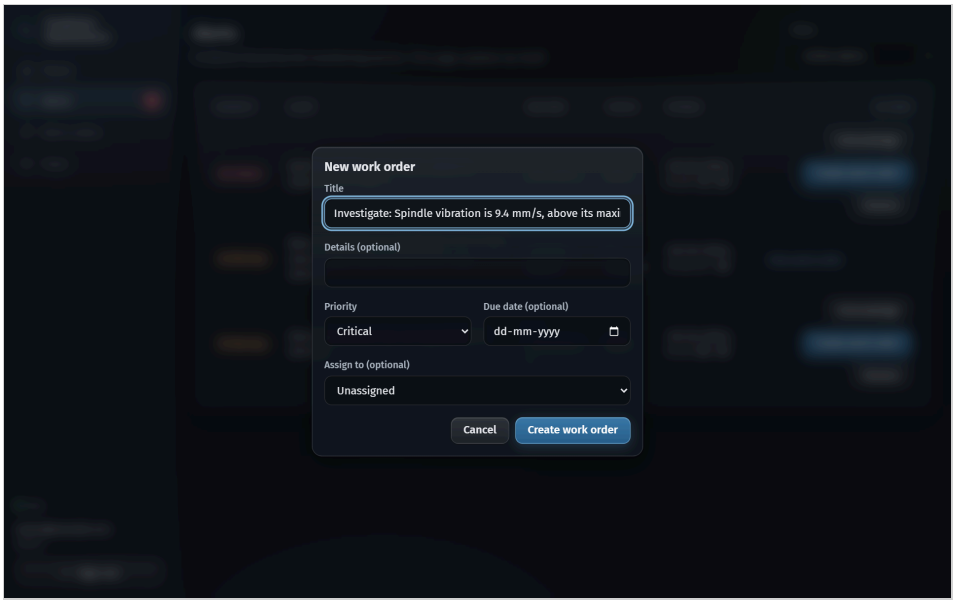


Figure 5: Work Order List with Priority, Assignee and Due Date

Shows the open and in-progress work orders with priority, machine, status, assignee and due date, and the actions to start, complete or cancel a job:

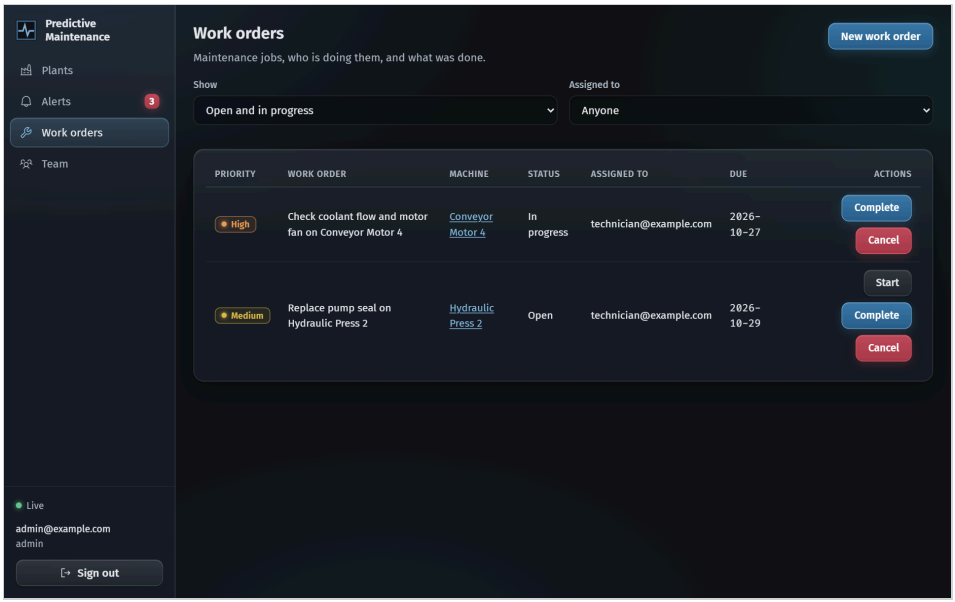
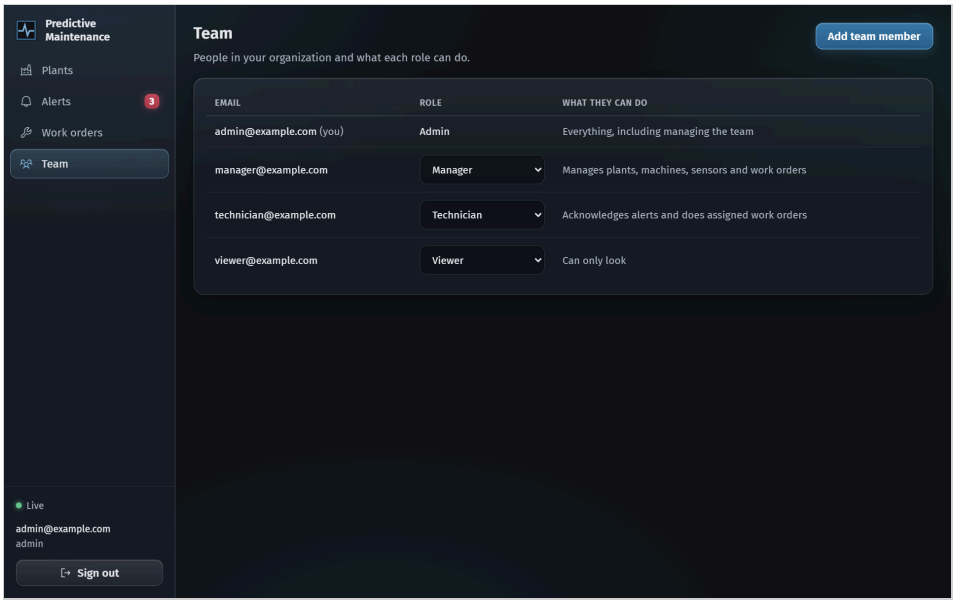


Figure 6: Team Management and Role-Based Permissions

Lists the members of the organization with their roles and what each role can do, and lets an admin add members or change roles:



REFERENCES

[1] A. K. S. Jardine, D. Lin, and D. Banjevic, "A review on machinery diagnostics and prognostics implemented on a condition-based maintenance," Mechanical Systems and Signal Processing, vol. 20, no. 7, pp. 1483-1510, 2006.

[2] R. K. Mobley, An Introduction to Predictive Maintenance, 2nd ed. Amsterdam, Netherlands: Butterworth-Heinemann, 2002.

[3] J. Lee, B. Bagheri, and H.-A. Kao, "A cyber-physical systems architecture for Industry 4.0-based manufacturing systems," Manufacturing Letters, vol. 3, pp. 18-23, 2015.

[4] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, R. da P. Francisco, J. P. Basto, and S. G. S. Alcala, "A systematic literature review of machine learning methods applied to predictive maintenance," Computers & Industrial Engineering, vol. 137, Art. no. 106024, 2019.

[5] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a distributed messaging system for log processing," in Proc. NetDB Workshop, Athens, Greece, 2011.

[6] OASIS, "MQTT Version 5.0," OASIS Standard, Mar. 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[7] International Organization for Standardization, "ISO 20816-1:2016, Mechanical vibration: Measurement and evaluation of machine vibration, Part 1: General guidelines," Geneva, Switzerland, 2016.

[8] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.

[9] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[10] N. Provos and D. Mazieres, "A future-adaptable password scheme," in Proc. USENIX Annual Technical Conference, FREENIX Track, Monterey, CA, USA, 1999, pp. 81-91.

[11] I. Sommerville, Software Engineering, 10th ed. London, UK: Pearson Education, 2015.

[12] MongoDB Inc., "Time series collections," MongoDB Manual, 2026. [Online]. Available: https://www.mongodb.com/docs/manual/core/timeseries-collections/

[13] Apache Software Foundation, "Apache Kafka documentation," 2026. [Online]. Available: https://kafka.apache.org/documentation/

[14] Spring, "Spring Boot reference documentation," 2026. [Online]. Available: https://docs.spring.io/spring-boot/

[15] React Team, "React documentation," 2026. [Online]. Available: https://react.dev

[16] Socket.IO, "Socket.IO documentation," 2026. [Online]. Available: https://socket.io/docs/v4/